

Practical Experiment: Compare the response speed of open-source Large Language Models (LLMs) to find the fastest model for real-time applications

Meeting 6 PIP Package Refactoring

Presented by: Mariyan Vasilev Apostolov
Date: 01th of September 2026
Version: 1.0

Agenda

- Refactoring Definition
- Benefits to Refactor
- When to Refactor
- The Refactoring Cycle
- Common Refactoring Techniques
- Refactoring Best Practices



Refactoring Definition

Refactoring is the process of restructuring existing computer syntax without changing its external behavior.

In Simple Terms

- Improving the internal structure of your code
- Cleaning up without breaking functionality
- Fixing technical construction issues

Key Principles

- External behavior stays the same
- Internal structure improves
- Done in small, safe steps

Benefits to Refactor

Code Quality

- Easier to read and understand
- Reduced complexity
- Consistent style and patterns

Development Speed

- Faster to add new features
- Easier to fix bugs
- Less time spent deciphering old code

Team Collaboration

- New developers onboard faster
- Clearer code reviews
- Shared understanding of the codebase

Business Value

- Lower maintenance costs
- Faster time-to-market
- Reduced technical debt



5 When to Refactor

Refactor when:

- You're adding a new feature
- You're fixing a bug
- You're doing a code review

Best Times to Refactor

Scenario	When
Before adding features	Clean up the code so the new feature fits naturally
After a feature is complete	Clean up the mess you made during development
During code reviews	Identify and fix issues together
Regularly	Set aside 10-20% of sprint time for maintenance
When you see duplication	Fix it immediately (Don't Repeat Yourself)

The Refactoring Cycle



Refactoring Flow

Common Refactoring Techniques

Simplifying Methods

- Extract Method – Break long methods into smaller pieces
- Inline Method – Merge simple methods back into caller
- Replace Temp with Query – Use methods instead of temporary variables

Simplifying Code

- Extract Class – Split a class into focused classes
- Move Method – Move methods to where they belong

Cleaning Up

- Rename Variables/Methods – Use clear, descriptive names
- Remove Dead Code – Delete unused imports, variables, methods
- Replace Magic Numbers – Use constants with meaningful names

Organizing

- Encapsulate Field – Make fields private with getters/setters
- Replace Conditional with Polymorphism – Use OOP instead of complex conditionals
- Separate Query from Modifier – Don't mix reading and writing

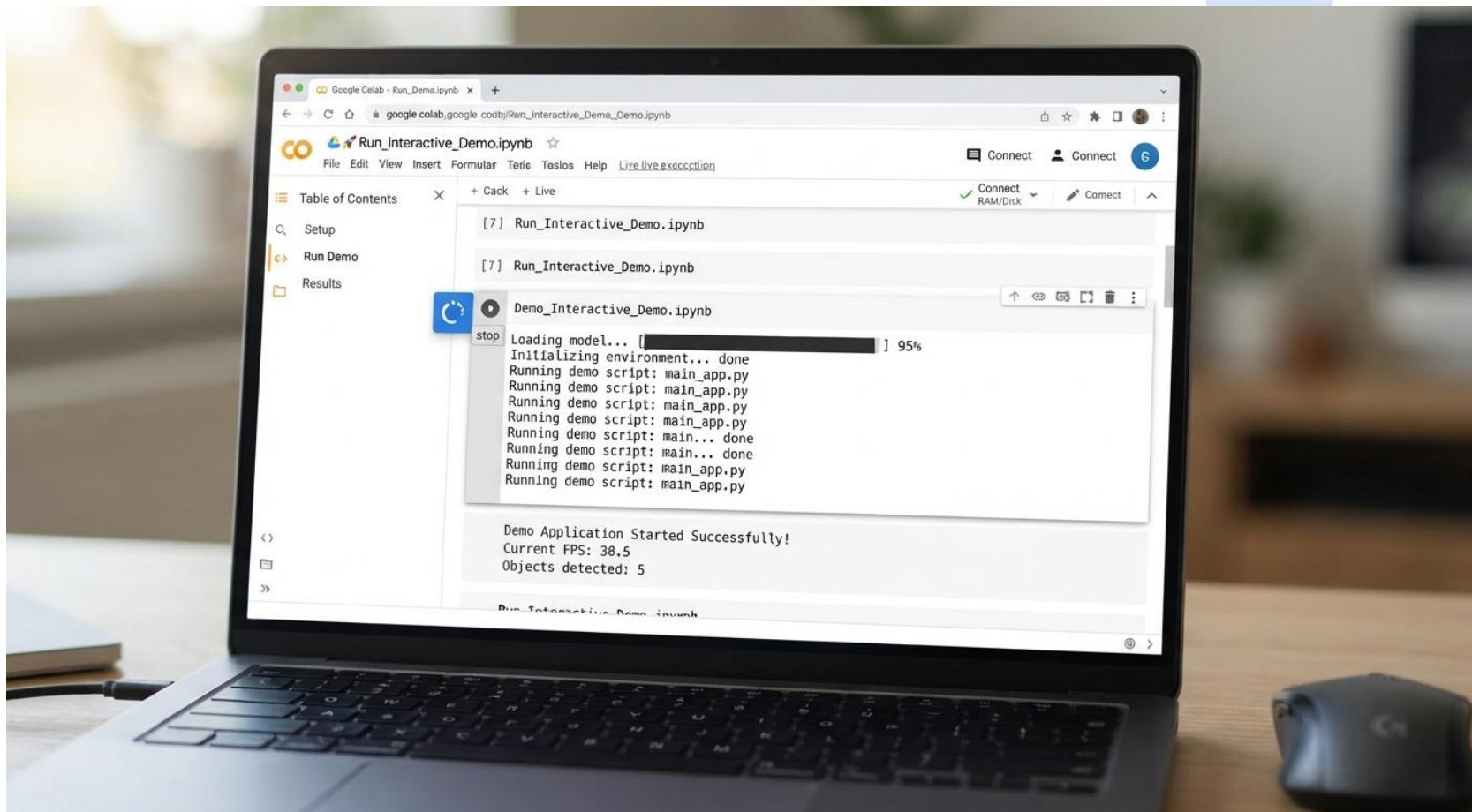
Refactoring Best Practices

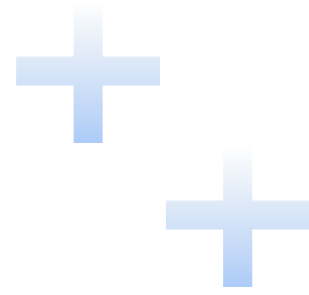


Do's	Don'ts
Make small, incremental changes	Refactor multiple things at once
Run tests after each change	Refactor without tests
Commit after each refactoring	Refactor production code directly
Use IDE refactoring tools	Refactor during a big feature release
Keep code reviews focused	Add new features during refactoring

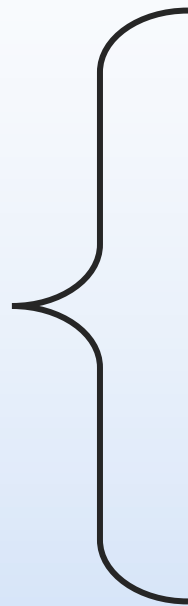
9

LIVE DEMO





Thank you



Mariyan Vasilev Apostolov

mariyan.apostolov89@gmail.com